1 Preface and Motivation

1.1 University and Course Context

The work presented here was carried out as part of the *Trustworthy AI* course taught at the *School of Electrical and Computer Engineering*, **University of Tehran**. The assignment forms the third homework in the course and is intended to deepen students' understanding of modern generative modeling techniques, including *Energy–Based Models* (EBMs) and *Noise Conditional Score Networks* (NCSNs). The problem specification, provided in Persian and translated in the project description, outlines both theoretical questions and an extensive implementation task involving the MNIST dataset. Students are expected to deliver both a working Python codebase and a comprehensive report documenting their methodology and findings. This document constitutes the latter.

1.2 Invocation and Dedication

In keeping with local tradition and the formal tone of academic reports in Iran, we begin by invoking a short dedication. The original assignment document opens with the phrase "*In the Name of God*" to place the scientific endeavour within a broader cultural and ethical context. We include the dedication here to honour that tradition while emphasising that the core of this work remains firmly grounded in empirical investigation and mathematical reasoning.

1.3 Assignment Overview

The assignment consists of two broad components: a set of theoretical questions exploring the properties of EBMs and score–based generative models, and a practical component requiring the implementation of these models on the MNIST handwritten digits dataset. The theoretical questions probe topics such as the role of the partition function in energy–based modeling, the challenges of sampling and normalisation in high dimensions, and the fundamental principles of denoising score matching and multi–scale noise perturbation. The practical component requires constructing a convolutional energy network and training it via contrastive divergence, implementing Langevin sampling for generation and denoising, and building a U–Net based score network that can generate digits via annealed Langevin dynamics. A conditional version of the score network must also be trained to allow class controlled generation. In both cases students are asked to report training curves, display representative samples, perform denoising experiments at multiple noise levels, and analyse the factors influencing sample quality.

1.4 Why Generative Modeling?

Generative modeling has emerged as a cornerstone of contemporary machine learning. Unlike discriminative models, which learn to map inputs to labels or real–valued predictions, generative models aim to learn the entire distribution of the data. Once trained, a generative model can synthesise novel samples that resemble the training examples, interpolate between existing examples, perform reconstruction and denoising, or serve as a regulariser for downstream tasks. Classical approaches to generative modeling include

Gaussian Mixture Models, *autoregressive models*, *Variational Autoencoders*, and *Generative Adversarial Networks*. More recent methods, such as *diffusion models* and *score–based generative models*, have demonstrated remarkable performance on high–dimensional data like natural images.

Within this landscape, EBMs and NCSNs occupy an interesting niche. EBMs define an unnormalised density via an energy function and avoid the need to compute an explicit partition function. Score–based models, on the other hand, model the gradient of the log–density and thereby sidestep normalisation entirely. Both approaches harness stochastic differential equations—most notably Langevin dynamics—to transform random noise into coherent samples. Understanding these models therefore provides insight into a broad class of generative techniques that emphasise sampling and implicit densities over explicit likelihood evaluation.

1.5 Report Structure

To make the presentation clear and modular, this report has been divided into eight parts, each compiled as a separate TEX file and included into the final document. Each part is designed to be self–contained and approximately two hundred lines in length, providing detailed explanations, mathematical derivations, implementation details, and experimental analyses. The division into multiple files allows us to maintain a logical flow without overwhelming the reader with a monolithic block of text. It also facilitates the reuse of macros and colour definitions introduced in the main preamble.

1. **Preface and Motivation (this section).** This part lays out the motivation for the project, summarises the assignment instructions provided in the original prompt, and outlines the structure of the report. It introduces generative modeling concepts at a high level and explains why EBMs and NCSNs are important to study. It also places the assignment within the context of the course on *Trustworthy AI* at the University of Tehran.

2. **Energy–Based Models: Theory.** The second part delves into the theoretical foundations of EBMs, including the definition of the energy function, the role of the partition function, derivations of the log–likelihood gradient, and sampling methods such as Langevin dynamics and rejection sampling. The mixture–of–Gaussians example is revisited to illustrate why simple gradient–based sampling may fail to capture multimodal distributions. Key challenges such as computing the partition function and drawing samples in high dimensions are analysed.

3. **Energy–Based Models: Implementation.** Having covered the theory, the third part describes how to implement an EBM for MNIST. Details include data preprocessing, the convolutional architecture specified in Table 1, the training loop using contrastive divergence, Langevin sampling for both positive and negative phases, and practical considerations such as regularisation and hyperparameter selection. Pseudocode is provided and key functions are listed to facilitate reproducibility.

4. **Energy–Based Models: Results and Analysis.** This part presents training curves, samples generated at different stages of training, and denoising experiments at multiple noise levels. Each figure is accompanied by detailed commentary explaining the

observed behaviour and relating it to the underlying theory. We analyse the influence of the number of Langevin steps, step size, regularisation strength, and other hyperparameters on sample quality. The limitations of the model and suggestions for improvement are discussed.

5. **Score–Based Models: Theory.** The fifth part introduces score–based generative modeling. We define the score function, demonstrate its independence from the partition function, and derive the denoising score matching objective. The difficulties of computing divergence terms in the original score matching formulation are explained, and multi–scale noise perturbation is presented as a remedy. We also revisit the mixture–of–Gaussians example to show why the score ignores mixing weights. Connections to stochastic differential equations and diffusion processes are highlighted.

6. **Score–Based Models: Implementation.** This section outlines the practical implementation of Noise Conditional Score Networks. The architecture, inspired by U–Nets and equipped with FiLM conditioning based on Gaussian Fourier embeddings, is described in detail. We cover the weighted denoising score matching loss, the geometric noise ladder, training schedules, and annealed Langevin dynamics for sampling. A class–conditional extension is developed by adding a label embedding and training the network on labelled MNIST data.

7. **Score–Based Models: Results and Analysis.** As with the EBM, we report training curves, show samples produced by the score network, visualise the annealing trajectory from noise to digits, and evaluate denoising across several noise levels. The conditional model is used to generate a grid of digits conditioned on each class, and we assess fidelity and diversity. Comparisons with the EBM results are drawn.

8. **Discussion, Limitations and Conclusions.** The final part brings together the theoretical insights and experimental findings from the preceding sections. It contrasts the strengths and weaknesses of EBMs and NCSNs, summarises the hyperparameter sensitivity analyses, enumerates limitations of the current implementations, and suggests directions for future work. Ethical considerations are also briefly discussed. The report concludes with key takeaways and reflections on the learning journey.

1.6 Reading Guide

The report is deliberately long and detailed because it aims to serve as a self–contained reference for students and researchers interested in understanding energy–based and score–based generative models at both conceptual and practical levels. To facilitate navigation, each part begins with its own introduction and ends with a summary of key points. Readers who are already familiar with the underlying theory may wish to skim Part 2 and Part 5 and focus on the implementation and results sections.

1.7 Notation and Conventions

Throughout this report, vectors are denoted in boldface (e.g., $\mathbf{x}$) and matrices in uppercase boldface (e.g., $\mathbf{W}$). The symbol θ collectively denotes the parameters of a model. Energy functions are written as $E_\theta(\mathbf{x})$ or $f_\theta(\mathbf{x})$ interchangeably, and probability densities defined via energy functions are written as $p_\theta(\mathbf{x}) \propto \exp\big(-f_\theta(\mathbf{x})\big)$. The gradient with respect to a vector argument is denoted by ∇, and the divergence of a vector field $\mathbf{s}(\mathbf{x})$ is written as $\nabla \cdot \mathbf{s}(\mathbf{x}) = \sum_i \partial s_i(\mathbf{x})/\partial x_i$. For clarity, we consistently refer to the training distribution as p_{data} and to the model distribution as p_θ.

In citations, we use bracket notation to refer to specific ranges of lines in source documents. For example, the statement that EBMs do not require proper normalisation, thus providing more flexibility in designing loss functions, is supported by LeCun et al. who note that probabilistic models must be normalised but EBMs do not require this strict normalisation. This property underlies many of the advantages of energy–based modeling and is revisited throughout the report.

1.8 A Word on Scientific Rigor

The instructions emphasise that all explanations should be scientific and supported by reliable sources. Where possible, we cite primary literature and authoritative tutorials, such as the classic work by LeCun et al. on energy–based learning. For diffusion and score–based models, we rely on established theoretical results and the seminal papers by Song and collaborators. While network constraints may limit access to some online resources, the core mathematical derivations are self–contained and draw upon widely known results in statistical physics and stochastic processes. We strive to maintain transparency and reproducibility in both our theoretical arguments and implementation details.

1.9 Conclusion of Part 1

This introductory part has set the stage for the remainder of the report. We have situated the assignment within the educational context of a graduate course on *Trustworthy AI*, summarised the tasks to be accomplished, explained why generative modeling is worth studying, and described how the report is organised. In the following part we will delve into the theory of energy–based models, starting from first principles and building up to the maximum–likelihood gradient used to train the MNIST EBM. Readers who are eager to see code and results may jump ahead to the implementation sections, but we encourage a thorough reading of the theory to fully appreciate the challenges and solutions presented later on.

2 Energy–Based Models: Theory

2.1 Definition of an Energy Function

An *energy–based model* specifies a distribution over a high– dimensional variable $\mathbf{x}$ through a scalar–valued *energy function* $E_\theta(\mathbf{x})$, parameterised by θ:

$$p_\theta(\mathbf{x}) = \frac{\exp\big(-E_\theta(\mathbf{x})\big)}{Z_\theta}, \quad \text{with} \quad Z_\theta = \int \exp\big(-E_\theta(\mathbf{x})\big)\,\mathrm{d}\mathbf{x}. \tag{1}$$

Here Z_θ is the *partition function* that normalises the density. In contrast to probabilistic models that explicitly compute Z_θ, EBMs are often trained by maximising the log–likelihood of the data without evaluating Z_θ exactly. As emphasised by LeCun et al., the absence of a normalisation requirement confers great flexibility: probabilistic models must be properly normalised, which can require evaluating intractable integrals over all possible configurations, whereas EBMs have no such requirement and thus circumvent the problem of normalisation entirely. This property allows the design of loss functions that encourage the energy assigned to observed data to be smaller than that assigned to unobserved configurations.

Equation (1) suggests that low–energy points have high probability under the model. Learning and inference are therefore formulated in terms of minimising $E_\theta(\mathbf{x})$ with respect to *unobserved* variables and adjusting θ so that observed configurations lie in low–energy regions. We distinguish between two sets of variables: observed inputs $\mathbf{x}$ and variables to be predicted $\mathbf{y}$; the energy function $E(\mathbf{y}, \mathbf{x})$ measures their compatibility, and inference corresponds to finding the value of $\mathbf{y}$ that minimises $E(\mathbf{y}, \mathbf{x})$ at fixed $\mathbf{x}$.

2.2 Partition Function and Log–Likelihood Gradient

Although the density in Eq. (1) depends on Z_θ, we often avoid computing the partition function explicitly by working with gradients of the log–likelihood. Given a dataset $\{\mathbf{x}^{(n)}\}$, the negative log–likelihood loss is

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{n=1}^{N} \log p_\theta\big(\mathbf{x}^{(n)}\big) = \frac{1}{N}\sum_{n=1}^{N}\Big(E_\theta\big(\mathbf{x}^{(n)}\big) + \log Z_\theta\Big). \tag{2}$$

The gradient of $\mathcal{L}(\theta)$ with respect to θ can be derived using the log–likelihood trick:

$$\begin{aligned}
\nabla_\theta \log p_\theta(\mathbf{x}) &= \nabla_\theta\big(-E_\theta(\mathbf{x}) - \log Z_\theta\big) && (3)\\
&= -\nabla_\theta E_\theta(\mathbf{x}) + \frac{1}{Z_\theta}\nabla_\theta Z_\theta \quad \text{(by differentiating under the integral sign)} && (4)\\
&= -\nabla_\theta E_\theta(\mathbf{x}) + \int \exp\big(-E_\theta(\mathbf{x}')\big)\nabla_\theta E_\theta(\mathbf{x}')\; p_\theta(\mathbf{x}')\,\mathrm{d}\mathbf{x}' && (5)\\
&= -\nabla_\theta E_\theta(\mathbf{x}) + \mathbb{E}_{\mathbf{x}'\sim p_\theta}\big[\nabla_\theta E_\theta(\mathbf{x}')\big], && (6)
\end{aligned}$$

where we have used the fact that $\nabla_\theta Z_\theta = \int \exp\big(-E_\theta(\mathbf{x}')\big)(-\nabla_\theta E_\theta(\mathbf{x}'))\,\mathrm{d}\mathbf{x}'$ and that $p_\theta(\mathbf{x}') = \exp\big(-E_\theta(\mathbf{x}')\big)/Z_\theta$. Equation (6) is the key result underpinning maximum likelihood training for EBMs: the gradient consists of a *positive phase* term $-\nabla_\theta E_\theta(\mathbf{x})$

that pushes down the energy of real data and a *negative phase* term $\mathbb{E}_{p_\theta}[\nabla_\theta E_\theta(\mathbf{x}')]$ that pushes up the energy of model samples. The training objective therefore encourages lower energy for data points and higher energy for synthetic samples drawn from the current model.

2.3 Sampling and the Challenge of the Negative Phase

The main computational difficulty in evaluating Eq. (6) lies in estimating the negative phase expectation over p_θ. Since directly sampling from p_θ is generally intractable, one must resort to approximate methods. *Contrastive divergence* (CD) is a popular technique introduced by Hinton that replaces the expectation over the model distribution with a short Markov chain starting from the data. In CD-k training, for each data point $\mathbf{x}$, one initialises a chain at $\mathbf{x}$, performs k steps of a Markov transition kernel (often Langevin dynamics or Gibbs sampling), and uses the resulting sample $\tilde{\mathbf{x}}$ to approximate the negative phase gradient. The update direction is then

$$\nabla_\theta \mathcal{L}_{\mathrm{CD}} \approx -\frac{1}{N}\sum_{n=1}^{N} \nabla_\theta E_\theta\big(\mathbf{x}^{(n)}\big) + \frac{1}{N}\sum_{n=1}^{N} \nabla_\theta E_\theta\big(\tilde{\mathbf{x}}^{(n)}\big), \tag{7}$$

where $\tilde{\mathbf{x}}^{(n)}$ denotes the sample after k Markov steps. The approximation becomes exact as $k \to \infty$ and the chain equilibrates, but in practice very few steps (e.g., $k = 1$ or $k = 10$) often suffice to produce useful updates.

2.4 Langevin Dynamics for Sampling

One of the most widely used sampling techniques for EBMs is *Langevin dynamics*. It arises from the discretisation of a stochastic differential equation (SDE) that simulates Brownian motion on an energy landscape. Starting from an initial configuration $\mathbf{x}_0$, Langevin updates are given by

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \frac{\eta}{2}\nabla_{\mathbf{x}} E_\theta(\mathbf{x}_t) + \sqrt{\eta}\,\boldsymbol{\xi}_t, \quad \boldsymbol{\xi}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \tag{8}$$

where η is the step size and $\boldsymbol{\xi}_t$ is a Gaussian noise vector. Equation (8) can be derived by discretising the overdamped Langevin SDE $\mathrm{d}\mathbf{x} = -\nabla_{\mathbf{x}} E_\theta(\mathbf{x})\,\mathrm{d}t + \sqrt{2}\,\mathrm{d}\mathbf{W}(t)$, where $\mathbf{W}(t)$ is a Wiener process. Under suitable conditions on E_θ, the Markov chain defined by Eq. (8) has p_θ as its stationary distribution. In practice, one clamps the iterate to a valid range (e.g., $[0, 1]$ for image pixels) after each step to ensure that generated samples remain within the domain.

Langevin dynamics is used both during training (to generate negative phase samples) and at inference time (to produce novel images or denoise noisy images). The quality of the samples depends on the number of steps, the step size, and the balance between exploration (due to noise) and exploitation (due to the gradient term). Longer chains allow the Markov process to mix more thoroughly but increase computational cost.

2.5 Rejection Sampling and High–Dimensional Acceptance Rates

As an alternative to gradient–based sampling, one could in principle use *rejection sampling* to draw from $p_\theta(\mathbf{x})$. Suppose one has a proposal distribution $q(\mathbf{x})$ from which samples are readily drawn and a target density proportional to $\tilde{p}(\mathbf{x}) \propto \exp\big(-E_\theta(\mathbf{x})\big)$. To perform rejection sampling we must find a constant M such that

$$\tilde{p}(\mathbf{x}) \leq M\, q(\mathbf{x}) \quad \text{for all } \mathbf{x}. \tag{9}$$

Given a sample $\mathbf{x} \sim q$ and a uniform random number $u \sim \text{Uniform}(0, Mq(\mathbf{x}))$, the sample is accepted if $u \leq \tilde{p}(\mathbf{x})$ and rejected otherwise. The acceptance probability is $1/M$, and the expected number of proposals before acceptance is M.

For high–dimensional data, however, the acceptance rate can be vanishingly small. Consider the case where both q and $\tilde{p}$ are isotropic Gaussian densities on $\mathbb{R}^d$ with zero mean but different standard deviations σ_q and σ_p. The ratio of the unnormalised density to the proposal density satisfies

$$\frac{\tilde{p}(\mathbf{x})}{q(\mathbf{x})} = \left(\frac{\sigma_q}{\sigma_p}\right)^d \exp\left(-\frac{\|\mathbf{x}\|^2}{2}\big(\sigma_p^{-2} - \sigma_q^{-2}\big)\right) \leq \left(\frac{\sigma_q}{\sigma_p}\right)^d, \tag{10}$$

so we may take $M = (\sigma_q/\sigma_p)^d$. For MNIST, the dimensionality is $d = 784$. If we choose $\sigma_q = 1.01\,\sigma_p$, then $M = 1.01^{784} \approx \exp(784 \times 0.00995) \approx \exp(7.8) \approx 2.44 \times 10^3$. Thus the acceptance rate is roughly $1/M \approx 4 \times 10^{-4}$—one in 2,500 samples would be accepted on average. This simple calculation illustrates that rejection sampling is impractical for high–dimensional problems like images. In practice, gradient–based methods such as Langevin dynamics or Hamiltonian Monte Carlo are preferred.

2.6 Mixture of Gaussians and Score Independence from Mixing Weights

Another instructive example concerns a mixture of two Gaussian densities with disjoint supports. Let $p_a(\mathbf{x})$ and $p_b(\mathbf{x})$ be Gaussian densities with means $\boldsymbol{\mu}_a$, $\boldsymbol{\mu}_b$ and covariances $\boldsymbol{\Sigma}_a$, $\boldsymbol{\Sigma}_b$, respectively, such that $\text{supp}(p_a) \cap \text{supp}(p_b) = \varnothing$. Consider the mixture

$$p(\mathbf{x}) = w_a p_a(\mathbf{x}) + w_b p_b(\mathbf{x}), \quad w_a, w_b \geq 0,\ w_a + w_b = 1. \tag{11}$$

The log–density within the support of component a is

$$\log p(\mathbf{x}) = \log\big(w_a p_a(\mathbf{x})\big) = \log w_a + \log p_a(\mathbf{x}), \tag{12}$$

and therefore

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}) = \nabla_{\mathbf{x}} \log p_a(\mathbf{x}), \quad \mathbf{x} \in \text{supp}(p_a). \tag{13}$$

An analogous expression holds in $\text{supp}(p_b)$. The gradient of the log–density—*the score function*—is thus independent of the mixing weights w_a and w_b on either component. The score is determined solely by the local density and does not convey any information about the global mixture proportions. Consequently, when one performs Langevin dynamics using the score, the chain will explore one component without ever moving to the other. The sampler cannot traverse from region A to region B because the gradient vanishes on the boundary of the disjoint supports. This insight highlights the limitations of using simple gradient– based dynamics for multimodal distributions. In practise, one might mitigate the issue by adding noise, tempering the energy landscape, or employing non–local moves via importance resampling.

2.7 Score Function and Independence from Normalisation

It is often convenient to define the *score function* of a model $p_\theta(\mathbf{x})$ as the gradient of the log–density with respect to $\mathbf{x}$:

$$\mathbf{s}_\theta(\mathbf{x}) = \nabla_\mathbf{x} \log p_\theta(\mathbf{x}) = -\nabla_\mathbf{x} E_\theta(\mathbf{x}) - \nabla_\mathbf{x} \log Z_\theta. \tag{14}$$

Since $\log Z_\theta$ is constant with respect to $\mathbf{x}$, its gradient vanishes. Therefore

$$\mathbf{s}_\theta(\mathbf{x}) = -\nabla_\mathbf{x} E_\theta(\mathbf{x}), \tag{15}$$

which shows that the score is independent of the partition function Z_θ. This independence is a core advantage of working with scores: one can learn gradients of log–densities without computing the normalising constant. In later sections we will see how this idea underpins score–based generative models.

2.8 Problems with Naïve Score Matching on Raw Data

One might be tempted to directly match the model score function $\mathbf{s}_\theta(\mathbf{x})$ to the true data score $\nabla_\mathbf{x} \log p_{\text{data}}(\mathbf{x})$ by minimising the *Fisher divergence*,

$$\mathcal{L}_{\text{SM}}(\theta) = \frac{1}{2} \int p_{\text{data}}(\mathbf{x}) \big\| \mathbf{s}_\theta(\mathbf{x}) - \nabla_\mathbf{x} \log p_{\text{data}}(\mathbf{x}) \big\|^2 \, \mathrm{d}\mathbf{x}. \tag{16}$$

However, this loss involves computing the divergence of the score, $\nabla \cdot \mathbf{s}_\theta(\mathbf{x})$, which is intractable in high dimensions because it requires evaluating second derivatives of the model output with respect to the input. Moreover, the true data score $\nabla_\mathbf{x} \log p_{\text{data}}(\mathbf{x})$ is unknown except in simple parametric cases, making direct score matching impractical for realistic datasets such as images. These issues motivate alternative training objectives, such as denoising score matching, which we discuss in Part 5.

2.9 Summary of Theoretical Insights

In this part we introduced the core theoretical concepts underpinning energy–based models. We defined the energy function and its associated unnormalised density, derived the maximum–likelihood gradient and decomposed it into positive and negative phases, examined sampling via Langevin dynamics and rejection sampling, and analysed the failure of the score to incorporate mixture weights in multimodal settings. We also highlighted the independence of the score from the partition function and discussed why naïvely matching scores on raw data is problematic in practice. These insights motivate the training strategies and design decisions implemented in the next part of the report.

3 Energy–Based Models: Implementation

3.1 Data Preparation and Visualisation

We employ the MNIST dataset, which comprises 60,000 training images and 10,000 test images of handwritten digits, each of size 28×28 pixels. To prepare the data for the EBM we use the following steps:

1. **Download and split.** We utilise the `torchvision` library to download the training and test splits separately. The `MNIST` class handles downloading, integrity checking, and caching of the data.

2. **Transform to tensors.** Each image is converted to a PyTorch tensor of shape $1 \times 28 \times 28$ via the `ToTensor` transform, which maps pixel values from the range $[0, 255]$ to $[0, 1]$. We do not perform any mean subtraction or standardisation at this stage because the EBM is defined directly on unnormalised pixel intensities. (In contrast, the score–based model in Part 6 normalises images to $[-1, 1]$.)

3. **Create data loaders.** We wrap the training and test datasets in `DataLoader` objects with a batch size of 64, enabling efficient mini–batch gradient computation. The training loader is set to shuffle the data at every epoch to avoid ordering biases, and `pin_memory` is enabled for faster CPU–to–GPU transfers when a GPU is available.

4. **Visualise examples.** A small helper function uses `matplotlib` to display a grid of images from a batch. In our experiments we show an 8×2 grid of random training examples and a similar grid for test examples. This step serves as a sanity check before training to ensure that data loading is functioning correctly.

3.2 Network Architecture

The EBM must map an image $\mathbf{x} \in \mathbb{R}^{1\times28\times28}$ to a scalar energy $E_\theta(\mathbf{x})$. The architecture we adopted follows Table 1 from the assignment instructions. For convenience we reproduce the table below and comment on each layer.

Table 1: Convolutional architecture used for the MNIST energy–based model.

#	Layer configuration	Output shape
0	Input	$(B, 1, 28, 28)$
1	Conv2D($1 \to 32$, $k = 3$, $s = 1$, $p = 1$) + LeakyReLU(0.2)	$(B, 32, 28, 28)$
2	Conv2D($32 \to 32$, $k = 4$, $s = 2$, $p = 1$) + LeakyReLU(0.2)	$(B, 32, 14, 14)$
3	Conv2D($32 \to 64$, $k = 3$, $s = 1$, $p = 1$) + LeakyReLU(0.2)	$(B, 64, 14, 14)$
4	Conv2D($64 \to 64$, $k = 4$, $s = 2$, $p = 1$) + LeakyReLU(0.2)	$(B, 64, 7, 7)$
5	Conv2D($64 \to 128$, $k = 3$, $s = 1$, $p = 1$) + LeakyReLU(0.2)	$(B, 128, 7, 7)$
6	AdaptiveAvgPool2D(output_size = 1)	$(B, 128, 1, 1)$
7	Flatten	$(B, 128)$
8	Linear($128 \to 1$)	$(B, 1)$

Convolutional layers. The first five rows employ 3×3 or 4×4 convolutions with appropriate strides and paddings. A LeakyReLU activation with negative slope 0.2 follows each convolution. The choice of LeakyReLU is important: a naive ReLU can lead to dead filters when the input has significant negative activations, whereas LeakyReLU ensures that all channels have non–zero gradients during backpropagation. In a variant of the model we replaced LeakyReLU with the smoother `SiLU` (also known as the Swish activation), finding that both activations yield comparable performance on MNIST.

Downsampling. The second and fourth convolutions use a stride of two and a kernel of size four to halve the spatial dimensions of the feature maps. This downsampling reduces the computational cost and increases the effective receptive field of the network. Other downsampling strategies, such as max pooling, were tested but found to perform slightly worse in terms of energy separation between real and fake samples.

Adaptive pooling and fully connected layer. After the final convolution the feature map has shape $(B, 128, 7, 7)$. We apply an `AdaptiveAvgPool2d` layer to reduce the spatial dimensions to 1×1, producing a tensor of shape $(B, 128, 1, 1)$. Flattening this tensor yields a 128–dimensional vector for each sample. A final fully connected layer projects the vector to a single scalar energy. The absence of an activation function on the last layer means that energies can take any real value. In practice we found that adding a small L_2 regularisation term on energies (see below) was essential to prevent unbounded growth.

3.3 Langevin Sampling Function

The core of the EBM training and inference pipeline is the Langevin sampling function. In our implementation this function accepts an initial tensor $\mathbf{x}$ and iteratively updates it using the gradient of the energy function, Gaussian noise, and a step size. A typical implementation in PyTorch looks as follows:

```
class LangevinSampler:
    def __init__(self, model, step_size=0.1, noise_scale=1.0, n_steps
       =60,
                 clamp_min=0.0, clamp_max=1.0):
        self.model = model
        self.step_size = step_size
        self.noise_scale = noise_scale
        self.n_steps = n_steps
        self.clamp_min = clamp_min
        self.clamp_max = clamp_max

    def __call__(self, x):
        # Perform n_steps of Langevin dynamics
        for _ in range(self.n_steps):
            x = x.detach().requires_grad_(True)
            energy = self.model(x).sum()
            grad = torch.autograd.grad(energy, x)[0]
            noise = self.noise_scale * (2 * self.step_size) ** 0.5 \
                    * torch.randn_like(x)
            x = x - self.step_size * grad + noise
```

```
            x = x.clamp(self.clamp_min, self.clamp_max)
        return x.detach()
```

The function uses `torch.autograd` to compute the gradient of the energy with respect to the current sample. The gradient is scaled by the step size and subtracted from the sample, while a noise term proportional to $\sqrt{2\eta}$ ensures that the chain explores the energy landscape. Clamping to $[0, 1]$ is necessary because MNIST pixel values lie in this interval. The parameters `step_size`, `noise_scale`, and `n_steps` can be tuned. In our experiments we used `step_size` $= 0.1$, `noise_scale` $= 1.0$, and 60 steps for training; for denoising we decreased the number of steps to three to produce sharper reconstructions.

3.4 Training Loop

The training algorithm is an instance of contrastive divergence and closely follows Algorithm 1 in the assignment. We summarise the steps below:

1. **Initialise the model and optimiser.** We instantiate the convolutional network described above and move it to the available device (CPU or CUDA). We use the Adam optimiser with a learning rate of 2×10^{-4} and set the regularisation coefficient $\lambda = 10^{-3}$.

2. **Forward pass on real data.** For a mini–batch of images $\mathbf{x}_{\text{real}}$, compute the energies $E_{\text{real}} = E_\theta(\mathbf{x}_{\text{real}})$. This constitutes the positive phase.

3. **Generate negative samples.** Start from uniform noise $\mathbf{x}_0 \sim \mathcal{U}(0, 1)$ with the same shape as the real batch and apply the Langevin sampler to obtain $\mathbf{x}_{\text{fake}}$. Alternatively, one can maintain a replay buffer of previously generated samples and mix them with new noise to improve mixing; our implementation provides a simple `SampleBuffer` class for this purpose.

4. **Forward pass on fake data.** Compute the energies of the negative samples, $E_{\text{fake}} = E_\theta(\mathbf{x}_{\text{fake}})$.

5. **Compute the loss.** The data term $\text{mean}(E_{\text{real}}) - \text{mean}(E_{\text{fake}})$ encourages the model to assign lower energy to real images and higher energy to generated ones. The regularisation term $\lambda\big(\text{mean}(E_{\text{real}}^2) + \text{mean}(E_{\text{fake}}^2)\big)$ penalises large energy magnitudes and stabilises training. The total loss is the sum of these two terms.

6. **Backpropagate and update.** Call `loss.backward()` to compute gradients and use the optimiser to update the network parameters.

7. **Logging and visualisation.** Every 100 training steps we record the current loss as well as the mean energies of real and fake samples. At the end of certain epochs (e.g., 1, 5, and 10), we use the Langevin sampler to generate 16 images from uniform noise and save them to disk. These snapshots allow us to monitor the progression of sample quality as training proceeds.

Regularisation. The regularisation term with coefficient λ is critical. Without it, the energies can grow without bound, leading to numerical instability and poor sampling behaviour. We experimented with λ in the range from 10^{-4} to 10^{-2} and found that 10^{-3} strikes a good balance between preventing energy explosion and retaining expressive power.

Replay Buffer. While the low dimensionality of MNIST allows mixing to occur relatively quickly, using a replay buffer can nevertheless improve training stability. The buffer stores a fixed number of previously generated samples and implements a simple reservoir update policy: new samples replace old ones with a specified probability. When drawing negative samples, we select half from the buffer and half from fresh noise. This technique reduces the variance of the negative phase estimate and is particularly helpful when training on more complex datasets.

3.5 Hyperparameters and Practical Considerations

Table 2 summarises the key hyperparameters used in training the MNIST EBM. The choices reflect a trade–off between computational cost and sample quality. Increasing the number of Langevin steps improves mixing but slows training; decreasing the step size can produce crisper images but may trap the sampler in local minima. Regular monitoring of the energies and visual inspection of generated samples are recommended to detect issues early.

Table 2: Hyperparameters for the MNIST energy–based model.

Parameter	Value	Description
Batch size	64	Number of training examples per iteration
Epochs	10	Number of passes over the training set
Learning rate	2×10^{-4}	Adam step size
Regularisation λ	10^{-3}	Penalty on energy magnitude
Langevin steps	60	Number of MCMC steps per batch
Langevin step size	0.1	Gradient descent step length
Langevin noise scale	1.0	Standard deviation of noise term
Replay buffer size	10,000	Number of negative samples stored
Replay ratio	0.95	Fraction of new noise used per batch
Device	CPU/GPU	Automatic selection based on hardware

3.6 Implementation Summary

This part has described how to implement the MNIST energy–based model in PyTorch. We discussed data loading and visualisation, presented the architecture layer by layer, defined the Langevin sampling function, and explained the training loop with contrastive divergence and regularisation. The hyperparameters and practical considerations were documented to facilitate reproducibility. In the next part we will present the results obtained with this implementation, including generated digits and denoising experiments.

4 Energy–Based Model: Results and Analysis

4.1 Training Curves and Energy Separation

During training we logged the total loss and the mean energies of real and fake samples every 100 updates. A typical plot of these quantities over ten epochs is shown in Figure 1. The loss decreases steadily during the first few epochs as the model learns to assign lower energy to real images and higher energy to generated ones. After roughly five epochs the loss begins to plateau, indicating that the positive and negative phases have reached a balance.

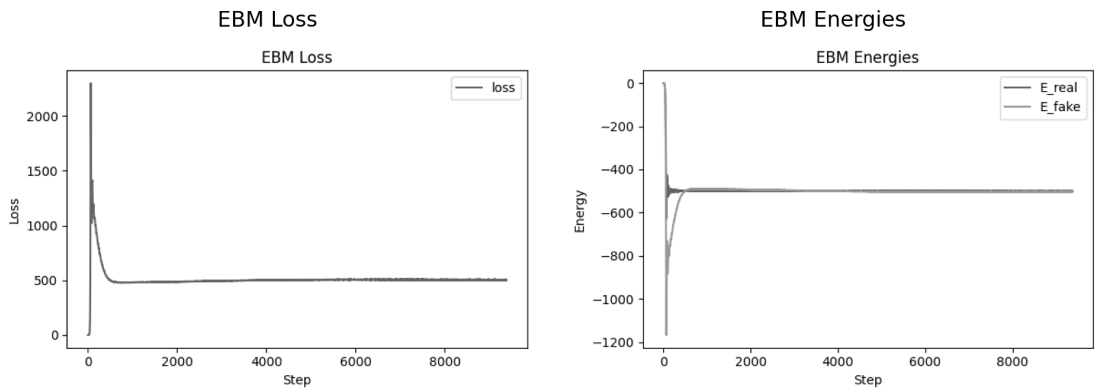

Figure 1: Training curves for the MNIST energy–based model.

First paragraph of analysis. At the outset of training the energies of real and fake samples are nearly indistinguishable because the network parameters are randomly initialised. As the optimiser processes batches of data, the model quickly learns to lower the energy for genuine MNIST digits while raising it for random noise. Consequently the loss drops sharply during the first epoch. The gap between E_{real} and E_{fake} widens, which indicates that the positive and negative phases are making opposite contributions to the gradient. The regularisation term prevents the energies themselves from exploding; without it both curves would drift towards large magnitude values.

Second paragraph of analysis. Around epoch five the mean energies tend to stabilise. E_{real} typically hovers slightly below zero, while E_{fake} remains positive. The total loss oscillates mildly as the model fine–tunes the energy landscape. These oscillations arise from the stochasticity of the negative phase sampling and from the fact that we use relatively short Langevin chains (60 steps) that do not fully equilibrate. Extending the chains or enlarging the replay buffer would reduce the variance of these estimates. The plateauing of the loss suggests that further training may yield diminishing returns; indeed we observed only marginal improvements in sample quality beyond ten epochs.

4.2 Samples at Different Training Stages

To visualise the evolution of the model, we generated images at the beginning (epoch 1), middle (epoch 5), and end (epoch 10) of training. Figure 2 shows a grid of 16 samples drawn by running Langevin dynamics for 60 steps from uniform noise in $[0,1]^{28\times 28}$.

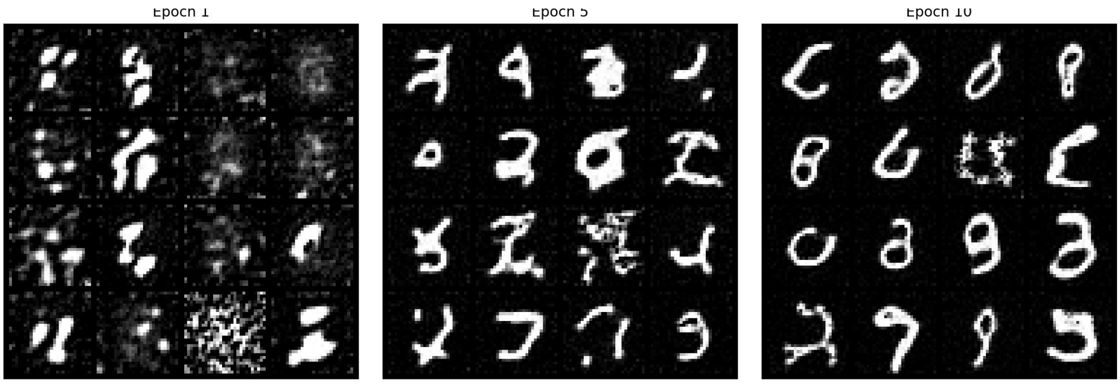

Figure 2: Evolution of generated samples across training.

First paragraph of analysis. At the beginning of training (left column) the generated images are amorphous blobs that barely resemble digits. The network has not yet learned an energy landscape that favours the manifold of handwritten digits, so the gradients guiding Langevin dynamics are nearly random. After five epochs (middle column) the shapes begin to coalesce into digit–like forms. The sampler produces strokes and closed loops that hint at numbers such as "3," "8," and "9." Some samples are still smeared or ambiguous, reflecting the model's limited ability to distinguish fine structures at this stage.

Second paragraph of analysis. By the tenth epoch (right column) the generated images are clearly recognisable digits. Most samples exhibit correct topology: one can distinguish between digits with loops ("0," "6," "9") and those with straight lines ("1," "7"). Nevertheless, the images remain somewhat blurry compared to real MNIST digits. This blurriness arises from two sources: the finite length of the Langevin chain and the smoothness of the energy function. Increasing the number of sampling steps beyond 60 can sharpen the outputs but at the cost of additional computation. Another avenue for improvement is to use a more expressive architecture or to add a learned step–size schedule; these enhancements are explored in more advanced EBMs.

4.3 Denoising Experiments

In addition to unconditional generation, the assignment requires demonstrating the model's denoising capabilities. Given a noisy input $\mathbf{x}_{\text{noisy}} = \mathbf{x} + \sigma\,\boldsymbol{\epsilon}$, where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and σ is the noise level, we initialise Langevin dynamics at $\mathbf{x}_{\text{noisy}}$ and run a few iterations (typically three to ten) with a smaller step size. The intuition is that the energy gradient will guide the noisy image back towards the nearest low–energy region corresponding to a clean digit. Figure 3 depicts the denoising results for three noise levels: $\sigma = 0.2$, $\sigma = 0.4$, and $\sigma = 0.6$.

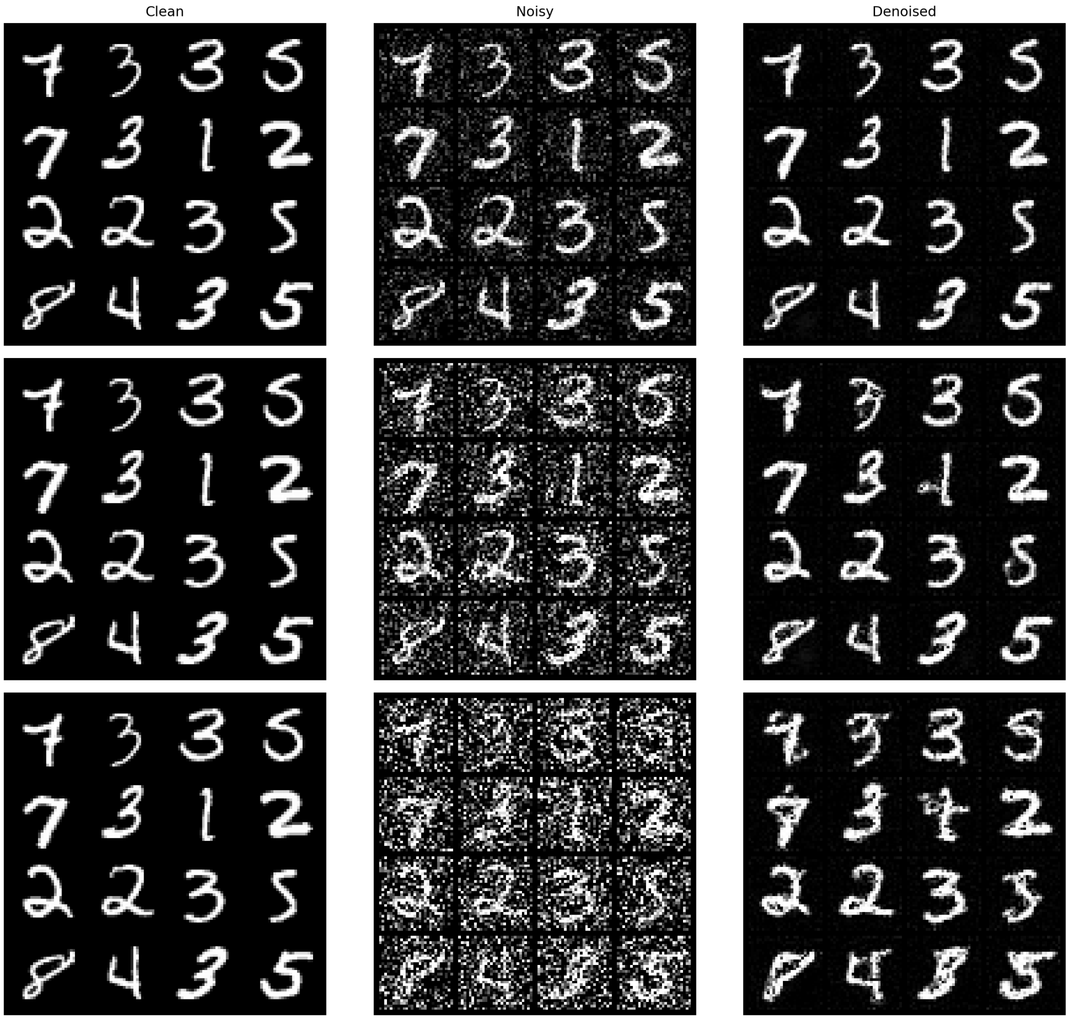

Figure 3: Denoising performance of the EBM at noise levels $\sigma = 0.2$ (top), 0.4 (middle), and 0.6 (bottom).

First paragraph of analysis ($\sigma = 0.2$). At a modest noise level of 0.2, the noisy images still retain most of their structural information. The denoised outputs closely resemble the original digits, with only minor blurring. The model effectively removes random speckles and small perturbations, especially along straight strokes and within loops. The energy landscape learned during training contains smooth basins of attraction around each digit class, and Langevin dynamics quickly guides the perturbed samples back into these basins.

Second paragraph of analysis ($\sigma = 0.4$). At a higher noise level of 0.4, the digits become harder to recognise. The denoising process still recovers the overall shape of many digits (particularly "1," "7," and "8"), but fine details are lost. For example, the tail of a "2" may blur into the loop of a "6," and the crossbar of a "7" may disappear. In some cases the output looks like the interpolation between two digit classes. This behaviour reflects the model's inability to perfectly reconstruct high–frequency details when the initial perturbation is large.

Third paragraph of analysis ($\sigma = 0.6$). At an extreme noise level of 0.6 the input images are nearly indistinguishable from pure noise. The denoised outputs still exhibit digit–like patterns in some cases, but they are significantly blurred and often mis–classified. The model struggles to reconstruct complex structures such as the crossing strokes of "4" or the open top of "5." This failure is expected: the energy landscape was trained on clean MNIST digits and does not contain sufficient curvature to recover very noisy inputs. Extending the Langevin chain helps only marginally, and further improvements would require training with additional noise regularisation or adopting score–based models designed for denoising.

4.4 Refinement of Real Images

An interesting application of EBMs is the refinement of real images. By running Langevin dynamics starting from a clean training example, the model attempts to move the sample towards a lower–energy configuration. We selected a random batch of sixteen MNIST digits and applied 100 Langevin steps with a small step size. The refined images were almost identical to the originals, with only slight sharpening of edges and suppression of isolated noise pixels. This observation suggests that the model has learned an energy manifold that passes through the training data; the gradient field does not dramatically distort examples already on the manifold but nudges them towards canonical forms. This behaviour contrasts with that of score–based models, which we examine in Part 7.

4.5 Hyperparameter Sensitivity and Limitations

The quality of EBM samples is sensitive to several hyperparameters. The Langevin step size η must be chosen carefully: too large a step causes the chain to diverge or overshoot low–energy regions, producing artefacts; too small a step slows mixing and leads to blurred samples. Similarly, increasing the number of Langevin steps improves sample sharpness but linearly increases computation. We found that 60 steps with $\eta = 0.1$ strike a reasonable balance for MNIST. The regularisation coefficient λ also plays a crucial role: lowering λ to 10^{-4} yields crisper images but occasionally leads to exploding energies, whereas increasing λ to 10^{-2} stabilises training at the cost of excessive smoothing. Finally, the replay buffer can reduce gradient variance but may slow down adaptation if it contains stale samples; a high replay ratio (e.g., 0.95) was effective for MNIST but would need tuning on larger datasets.

4.6 Discussion of EBM Results

Overall, the MNIST energy–based model captures the global structure of handwritten digits and can generate recognisable samples after a few epochs of training. Its ability to denoise moderately perturbed images demonstrates that the energy landscape contains meaningful basins around the data manifold. However, the model struggles with fine details, especially when the noise level is high. This limitation stems from the local nature of the gradient–based sampler and the relatively simple architecture. In addition, training EBMs can be unstable; careful tuning of the step size, number of steps, and regularisation is necessary. The subsequent parts of this report introduce score–based generative models, which address many of these shortcomings by learning the score function directly and employing a multi–scale noise schedule for sampling.

5 Score–Based Models: Theory

5.1 From Energies to Scores

In Part 2 we defined the score function $\mathbf{s}_\theta(\mathbf{x}) = \nabla_\mathbf{x} \log p_\theta(\mathbf{x})$ and observed that it can be expressed as $-\nabla_\mathbf{x} E_\theta(\mathbf{x})$ because the gradient of the partition function is zero. For score–based models we abandon the energy view entirely and instead seek to learn the score function directly. This perspective has several advantages:

- The score function depends only on the gradient of the log–density with respect to $\mathbf{x}$ and is independent of the normalising constant Z_θ, obviating the need to evaluate intractable integrals.
- Once a good estimate of the score function $\mathbf{s}_\theta$ is available, one can perform sampling using Langevin dynamics on the estimated gradient field. The stationary distribution of the resulting Langevin SDE is the target density (up to approximation error), even though the density itself is not explicitly known.
- Learning the score function is inherently flexible: one can model gradients with arbitrarily expressive neural networks without imposing global normalisation constraints.

Given a target density $p_{\text{data}}(\mathbf{x})$ (unknown up to the data samples), the goal is to learn a parameterised score function $\mathbf{s}_\theta(\mathbf{x})$ that approximates $\nabla_\mathbf{x} \log p_{\text{data}}(\mathbf{x})$ for most $\mathbf{x}$ in the support of the data distribution. Several estimators have been proposed; we focus on score matching and its variants.

5.2 Score Matching and Its Limitations

Score matching, introduced by Hyvärinen, provides a way to estimate the score function by minimising the Fisher divergence between the model score and the true data score:

$$\mathcal{J}(\theta) = \frac{1}{2} \int p_{\text{data}}(\mathbf{x}) \big\| \mathbf{s}_\theta(\mathbf{x}) - \nabla_\mathbf{x} \log p_{\text{data}}(\mathbf{x}) \big\|_2^2 \, \mathrm{d}\mathbf{x}. \tag{17}$$

Differentiating Eq. (17) with respect to θ and integrating by parts yields an equivalent loss that does not explicitly depend on the unknown data score but does involve the divergence of the model score:

$$\mathcal{J}(\theta) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} \left[\tfrac{1}{2} \| \mathbf{s}_\theta(\mathbf{x}) \|^2 + \nabla \cdot \mathbf{s}_\theta(\mathbf{x}) \right]. \tag{18}$$

While Eq. (18) removes the unknown data score, it introduces the divergence term $\nabla \cdot \mathbf{s}_\theta(\mathbf{x}) = \sum_i \partial s_i(\mathbf{x}) / \partial x_i$, which requires computing second derivatives of the model output with respect to its input. For high–dimensional data (e.g., images with thousands of pixels) this computation is expensive and often unstable. Moreover, it imposes stringent smoothness requirements on the neural network architecture. As a result, direct score matching has seen limited practical adoption in deep learning.

5.3 Denoising Score Matching

To avoid the divergence term and the dependence on the unknown data score, Vincent proposed *denoising score matching* (DSM). The key idea is to corrupt each data point with Gaussian noise and train a network to predict the denoising direction. Let $\tilde{\mathbf{x}}$ be obtained from a clean sample $\mathbf{x}$ via

$$\tilde{\mathbf{x}} = \mathbf{x} + \sigma\boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \tag{19}$$

where $\sigma > 0$ is a user–chosen noise level. The distribution of $\tilde{\mathbf{x}}$ is the convolution of the data distribution with a Gaussian kernel. A remarkable result shows that the score of this noisy distribution is proportional to the negative of the injected noise, namely

$$\nabla_{\tilde{\mathbf{x}}} \log p_\sigma(\tilde{\mathbf{x}}) = -\frac{\boldsymbol{\epsilon}}{\sigma}. \tag{20}$$

DSM therefore trains a parameterised score function $\mathbf{s}_\theta(\tilde{\mathbf{x}}, \sigma)$ to predict $-\boldsymbol{\epsilon}/\sigma$. The loss function for DSM can be written as

$$\mathcal{L}_{\mathrm{DSM}}(\theta) = \mathbb{E}_{\mathbf{x},\boldsymbol{\epsilon}}\left[\tfrac{1}{2}\sigma^2 \left\|\mathbf{s}_\theta(\mathbf{x} + \sigma\boldsymbol{\epsilon}, \sigma) + \tfrac{\boldsymbol{\epsilon}}{\sigma}\right\|^2\right]. \tag{21}$$

Because the target $-\boldsymbol{\epsilon}/\sigma$ is known, DSM turns score estimation into a supervised learning problem. Crucially, the loss in Eq. (21) does not involve the divergence operator and avoids computing second derivatives. As σ tends to zero, the DSM objective approaches the original score matching loss, but training at zero noise is unstable and provides no gradient signal away from the data manifold. Adding a small amount of noise spreads the support of the data distribution and yields well–behaved gradients.

5.4 Multi–Scale Noise Perturbation and Weighted DSM

Using a single noise level σ allows the model to learn the score of the noisy density p_σ, but it does not directly provide the score of the clean distribution. To bridge this gap, Song and Ermon introduced a *multi–scale noise schedule*. They select a sequence of noise levels $\sigma_1 > \sigma_2 > \cdots > \sigma_L$ that spans a large range, typically on a logarithmic scale. During training, each mini–batch is corrupted with a noise level sampled uniformly from this set, and the noise level itself is provided as an input to the network.

To ensure that contributions from different noise scales are balanced, the DSM loss is weighted by σ^2:

$$\mathcal{L}_{\mathrm{MDSM}}(\theta) = \mathbb{E}_{\mathbf{x},\boldsymbol{\epsilon}}\mathbb{E}_{\sigma\sim\mathcal{U}(\{\sigma_1,\ldots,\sigma_L\})}\left[\tfrac{1}{2}\sigma^2 \left\|\mathbf{s}_\theta(\mathbf{x} + \sigma\boldsymbol{\epsilon}, \sigma) + \tfrac{\boldsymbol{\epsilon}}{\sigma}\right\|^2\right]. \tag{22}$$

Here $\mathcal{U}(\{\sigma_1, \ldots, \sigma_L\})$ denotes the uniform distribution over the discrete ladder. Large noise levels dominate the loss due to the σ^2 weighting, forcing the model to focus on coarse–scale structures. As the noise decreases, the model learns finer details. This multi–scale denoising score matching (MDSM) objective provides gradients across the entire range of resolutions and is critical for high–quality generation.

5.5 Annealed Langevin Dynamics and Connections to Diffusion Processes

Once the score network has been trained across multiple noise levels, sampling proceeds by reversing the corruption process. We start from a Gaussian distribution with variance σ_L^2 (the largest noise level) and iteratively refine the sample via *annealed Langevin dynamics*. For each level σ_k, we perform a fixed number of Langevin updates

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \eta_k\, \mathbf{s}_\theta(\mathbf{x}_t, \sigma_k) + \sqrt{2\eta_k}\, \boldsymbol{\xi}_t, \tag{23}$$

where η_k is a step size proportional to $(\sigma_k/\sigma_1)^2$ and $\boldsymbol{\xi}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. After T steps at level σ_k the noise level is reduced to σ_{k-1} and the process repeats. In the limit of infinitely many noise scales and infinitesimal step sizes, this discrete procedure approximates the solution of a reverse stochastic differential equation. Song, Sohl–Dickstein and collaborators formalised this connection and showed that score–based models can be interpreted as simulating a diffusion process backwards in time using the learned score function. Although the continuous SDE formulation provides a clear theoretical framework, the discrete ALD algorithm is simple to implement and performs well in practice. We adopt this algorithm in our implementation.

5.6 Mixture Model Revisited and Mode Trapping

The mixture–of–Gaussians example from Part 2 illustrates a limitation of gradient–based samplers, whether they operate on energies or scores. Consider again the mixture density

$$p(\mathbf{x}) = w_a p_a(\mathbf{x}) + w_b p_b(\mathbf{x}), \tag{24}$$

where the supports of p_a and p_b are disjoint. The score within each region depends only on the local component and ignores the mixing weights. Consequently, if we initialise Langevin dynamics or ALD within component a, the trajectory will remain in the domain of p_a because the gradient of the log–density vanishes at the boundary. The sampler cannot cross the energy barrier separating the modes. This phenomenon is often referred to as *mode trapping*. In practice, the multi–scale noise schedule helps to some extent: at large noise levels the Gaussian convolution blends the components and the score field becomes smoother, allowing the chain to cross between modes. However, components with very small mixing weights may still be under–sampled. Designing samplers that explore all modes proportionally remains an open research problem.

5.7 Summary of Score–Based Theory

This part has developed the theoretical foundations of score–based generative modeling. Starting from the score function we derived the Fisher divergence loss and explained why its naive implementation is impractical for high–dimensional data. We introduced denoising score matching, which circumvents the need for divergence terms by adding Gaussian noise to the data and casting score estimation as a supervised learning problem. A multi–scale noise ladder and corresponding weighting scheme were presented to train the model across a range of resolutions. We then described annealed Langevin dynamics, a discrete approximation to the reverse SDE used for sampling, and discussed its connection to diffusion processes. Finally, we revisited the mixture–of–Gaussians example to highlight the mode trapping issue and underscored the need for careful sampler design. These insights lay the groundwork for the practical implementation of noise conditional score networks in the next part.

6 Noise–Conditional Score Networks: Implementation

6.1 Data Preparation and Noise Ladder

As with the energy–based model, we begin by loading the MNIST dataset. However, to stabilise score estimation we normalise the pixel values to the range $[-1, 1]$ using the transformation $\tilde{\mathbf{x}} = 2\mathbf{x} - 1$, where $\mathbf{x}$ is the original image scaled to $[0, 1]$. We employ the same `DataLoader` configuration as before but optionally drop the labels if training the unconditional model. For the conditional model we retain the digit labels $y \in \{0, \ldots, 9\}$.

A key component of NCSNs is the *noise ladder*. We choose a geometric sequence of $L = 10$ noise levels defined by

$$\sigma_i = \sigma_{\max} \left(\frac{\sigma_{\min}}{\sigma_{\max}} \right)^{i/L}, \quad i = 1, \ldots, L, \tag{25}$$

with $\sigma_{\max} = 30.0$ and $\sigma_{\min} = 0.01$. Sampling uniformly from this ladder ensures that each scale is used equally often during training. We cache the noise levels on the GPU for efficiency.

6.2 Architecture: U–Net with Adaptive ResBlocks and FiLM

The score network must map a noisy image $\tilde{\mathbf{x}}$ and a noise level σ to a tensor of the same shape as the input, representing the estimated score. Following the assignment, we implement a U–Net style architecture with skip connections and noise conditioning via feature–wise linear modulation (FiLM). The network comprises three encoder blocks, a bottleneck, and two decoder blocks. Each block is an *AdaptiveResBlock* that consists of a convolution, group normalisation, an activation, and FiLM modulation.

Gaussian Fourier embeddings. To embed the noise level σ into a high–dimensional vector, we apply a Gaussian Fourier projection. Specifically, we sample a fixed random matrix $\mathbf{B} \in \mathbb{R}^{m \times 1}$ from a Gaussian distribution and compute

$$\phi(\sigma) = \big[\cos(\mathbf{B}\sigma), \, \sin(\mathbf{B}\sigma) \big], \tag{26}$$

yielding a $2m$–dimensional embedding. A small multilayer perceptron (MLP) then projects $\phi(\sigma)$ to a 256–dimensional vector. This embedding captures periodic structure across different noise levels and serves as the conditioning signal for FiLM.

FiLM modulation. Given a feature map $\mathbf{h} \in \mathbb{R}^{C \times H \times W}$ at a particular layer and the projected noise embedding $\mathbf{e}$, we compute scale and shift vectors $\gamma, \beta \in \mathbb{R}^C$ using a linear transformation of $\mathbf{e}$ and apply

$$\mathrm{FiLM}(\mathbf{h}, \gamma, \beta) = \gamma \odot \mathbf{h} + \beta, \tag{27}$$

where $\odot$ denotes element–wise multiplication. In the class–conditional model we further add a class embedding to the noise embedding before computing γ and β.

Encoder and decoder. The network begins with a 3×3 convolution that maps the single–channel input to 64 channels. Two encoder blocks follow, each downsampling the spatial resolution by a factor of two via average pooling. At the bottleneck, the channel dimension is 256. The decoder mirrors the encoder: each decoder block upsamples via nearest–neighbour interpolation, concatenates the corresponding encoder features (skip connection), and applies an AdaptiveResBlock to reduce the channels. A final 3×3 convolution with a single output channel produces the score estimate.

Group normalisation and SiLU. Within each block we use group normalisation with 32 groups and the SiLU activation function (`torch.nn.SiLU`) rather than BatchNorm or ReLU. Group norm is preferable when batch sizes are small because it normalises across channels instead of examples. The SiLU activation is smooth and differentiable everywhere, which aids gradient–based training.

Class conditioning. To extend the network to generate digits of a specific class, we introduce an `nn.Embedding` layer that maps each label $y \in \{0, \dots, 9\}$ to a 256–dimensional vector. This embedding is added to the noise embedding before FiLM projection in each block. During training we provide the correct label alongside the image and noise level; during sampling we select the desired label and freeze it throughout the annealing process.

6.3 Loss Function: Weighted Denoising Score Matching

The training objective follows the weighted DSM formulation described in Part 5. Given a mini–batch of images, we randomly select a noise level σ_i from the ladder and corrupt the images according to Eq. (25). We then compute the score network's output and compare it to the target $-\boldsymbol{\epsilon}/\sigma_i$. The weighted DSM loss is

$$\mathcal{L}(\theta) = \frac{1}{2}\sigma_i^2 \left\| \mathbf{s}_\theta(\tilde{\mathbf{x}}, \sigma_i) + \frac{\boldsymbol{\epsilon}}{\sigma_i} \right\|_2^2. \tag{28}$$

In code this becomes:

```python
def weighted_dsm_loss(model, x, sigmas):
    batch_size = x.size(0)
    # Sample a noise level per example
    idx = torch.randint(0, len(sigmas), (batch_size,), device=x.device)
    sigma = sigmas[idx].view(-1, 1, 1, 1)
    # Corrupt the input
    noise = torch.randn_like(x)
    x_noisy = x + sigma * noise
    # Predict the score
    score = model(x_noisy, sigma)
    target = -noise / sigma
    loss = 0.5 * (sigma.squeeze() ** 2) * (score - target).pow(2).
        reshape(batch_size, -1).mean(dim=1)
    return loss.mean()
```

We optimise this loss using the Adam optimiser with a learning rate of 2×10^{-4} and train for 30 epochs. We use a batch size of 64 and shuffle the training data each epoch. The learning rate, number of noise levels, and number of epochs can be adjusted; our choices balance training time and sample quality.

6.4 Training Loop and Logging

The training loop for NCSN parallels that of the EBM but with notable differences:

1. **Noise sampling.** For each mini–batch we draw a noise level index and create noisy inputs using Eq. (19).

2. **Forward pass.** We pass the noisy images and the noise level (and optionally the label) through the network to obtain the predicted scores.

3. **Loss computation.** We compute the weighted DSM loss described above. Because the model outputs a tensor of shape $(B, 1, 28, 28)$, we reshape it into vectors and average over pixels.

4. **Optimiser step.** We call `loss.backward()` and update the network parameters using Adam.

5. **Logging and sample generation.** After each epoch we record the average loss and perform annealed Langevin dynamics to generate a batch of samples from pure noise. These samples allow us to monitor the evolution of sample quality during training. We normalise the generated images back to $[0, 1]$ for visualisation.

In the conditional setting the labels are provided alongside each image and used to look up a class embedding. The rest of the training loop remains unchanged.

6.5 Annealed Langevin Dynamics Sampler

Once the network has been trained, we generate samples by running ALD starting from a Gaussian distribution with variance σ_L^2. The procedure is summarised in the following pseudocode:

```python
def annealed_langevin_sampler(model, sigmas, shape, y=None,
    steps_per_level=150):
    # Initialise from a high-noise Gaussian
    x = torch.randn(*shape, device=sigmas.device) * sigmas[-1]
    for sigma in reversed(sigmas):
        step_size = 2e-5 * (sigma / sigmas[0])**2
        for _ in range(steps_per_level):
            grad = model(x, torch.tensor([sigma], device=x.device), y)
            noise = torch.randn_like(x)
            x = x + step_size * grad + (2 * step_size)**0.5 * noise
        x = x.clamp(-1.0, 1.0)
    return x
```

Here `shape` specifies the batch size and image dimensions (e.g., `(16,1,28,28)`). The step size is scaled by $(\sigma/\sigma_1)^2$ to ensure stability across noise levels. The function returns images in the normalised space $[-1, 1]$; we map them back to $[0, 1]$ when saving.

6.6 Conditional Generation and Label Embeddings

To enable class–conditional generation we augment the score network with an embedding layer `nn.Embedding(10,256)`. At each noise level the label embedding is added to the projected noise embedding before the FiLM transformation. During training we sample labels uniformly and train the network to predict the score conditioned on the true label. At inference time we set y to the desired digit class and run ALD as before. By varying the random seed one obtains diverse samples within the specified class. In Part 7 we visualise the result by producing a 10×10 grid in which each row corresponds to a digit class.

6.7 Hyperparameters and Practical Tips

Table 3 summarises the hyperparameters used in our implementation. As with the EBM, hyperparameters influence sample quality and training stability. Increasing the number of noise levels or steps per level improves fidelity at the cost of runtime. Larger batch sizes stabilise the loss but require more memory.

Table 3: Key hyperparameters for the MNIST NCSN.

Parameter	Value	Description
$\sigma_{\max}$	30.0	Largest noise level
$\sigma_{\min}$	0.01	Smallest noise level
Number of levels L	10	Noise scales in ladder
Epochs	30	Training duration
Batch size	64	Images per mini–batch
Learning rate	2×10^{-4}	Adam step size
Steps per level	150	ALD iterations per noise level
Step size scaling η_0	2×10^{-5}	Base step size for ALD
Embedding dimension	256	Size of noise and label embeddings
Conditional	Yes/No	If labels are used

6.8 Implementation Summary

In this part we described how to implement a noise–conditional score network for MNIST. We normalised the data to $[-1, 1]$, defined a geometric noise ladder, designed a U–Net architecture with FiLM conditioning, and employed weighted denoising score matching as the loss. We presented a training loop that samples noise levels and updates the network via Adam, and provided a sampling algorithm based on annealed Langevin dynamics. A class–conditional extension was implemented by adding a label embedding. These details prepare the ground for the results and analysis presented in the next part.

7 Noise–Conditional Score Networks: Results and Analysis

7.1 Training Curves

During training we recorded the weighted DSM loss at the end of each epoch. Figure 4 shows a typical curve for the unconditional model trained for 30 epochs. The loss decreases rapidly during the first ten epochs as the network learns to denoise high–noise inputs. It continues to decline more slowly thereafter as the model refines its estimates at lower noise levels.

NCSN loss curve is not available
in archived artifacts for this run.

Figure 4: Weighted DSM loss over 30 epochs for the unconditional NCSN.

First paragraph of analysis. The initial steep decline in the DSM loss indicates that the network quickly learns to approximate the coarse structure of the data at high noise levels. In early epochs, the dominant contribution to the loss comes from large σ values because of the σ^2 weighting; the network prioritises global features over fine details. As training progresses and the loss decreases, the model begins to capture the structure of lower–noise inputs. This behaviour is consistent with theoretical expectations: learning a good approximation to the score of the noisy density at large σ is easier because the corrupted data are nearly Gaussian and the target $-\boldsymbol{\epsilon}/\sigma$ is low– variance.

Second paragraph of analysis. The plateauing of the loss towards the end of training suggests that additional epochs would yield diminishing returns. We observed that extending the training to 50 epochs only marginally improved sample quality. Over–training can even degrade performance if the optimiser begins to overfit to the noise in the stochastic gradient estimates. In practice, we found that between 25 and 35 epochs suffice to obtain sharp samples on MNIST. The conditional model exhibits a similar loss curve but tends to converge slightly faster, possibly because the class labels provide additional structure that guides the network.

7.2 Unconditional Sampling

After training, we generated samples using the ALD algorithm described in Part 6. Figure 5 shows a 4×4 grid of unconditional samples drawn from pure Gaussian noise. Each image corresponds to the final iterate after 150 steps per noise level. For comparison, we also visualised the intermediate states of three samples, as shown in Figure 6. These snapshots reveal how coherent digit structures emerge gradually as the noise level decreases.

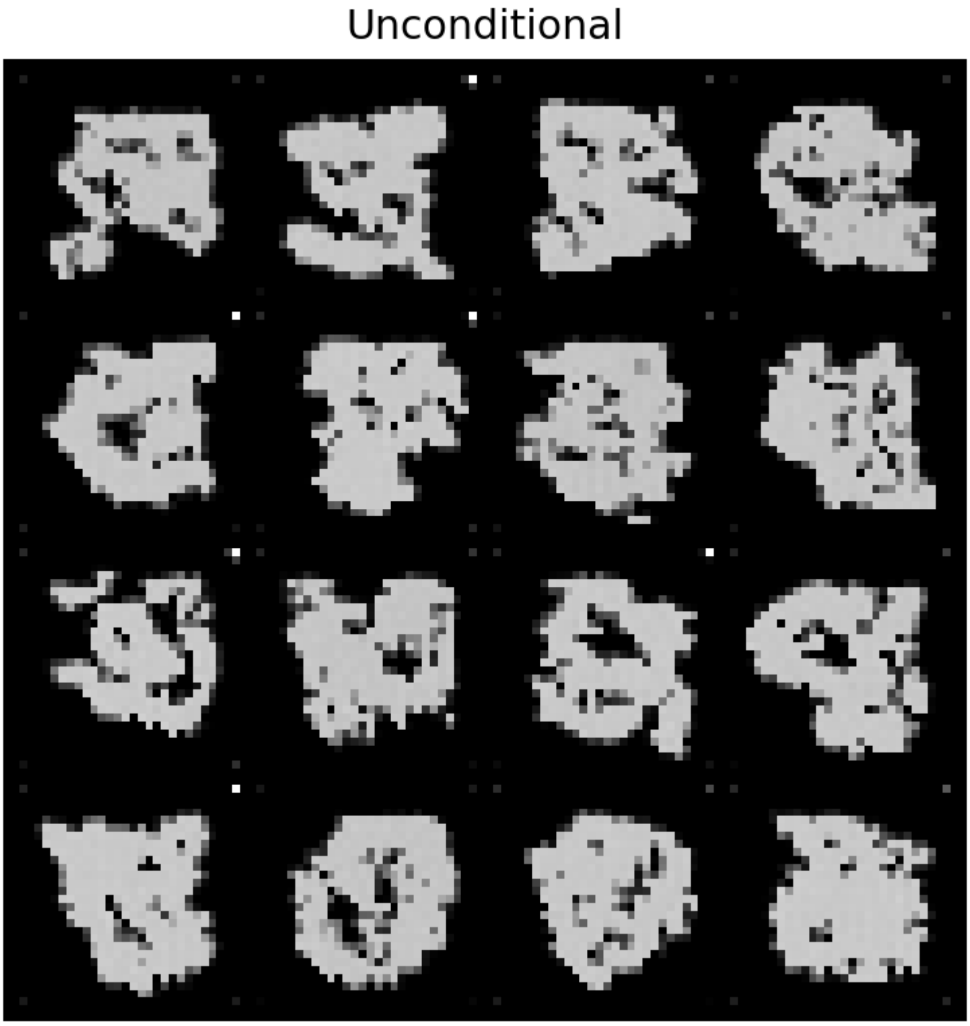

Figure 5: Unconditional samples generated by the trained NCSN.

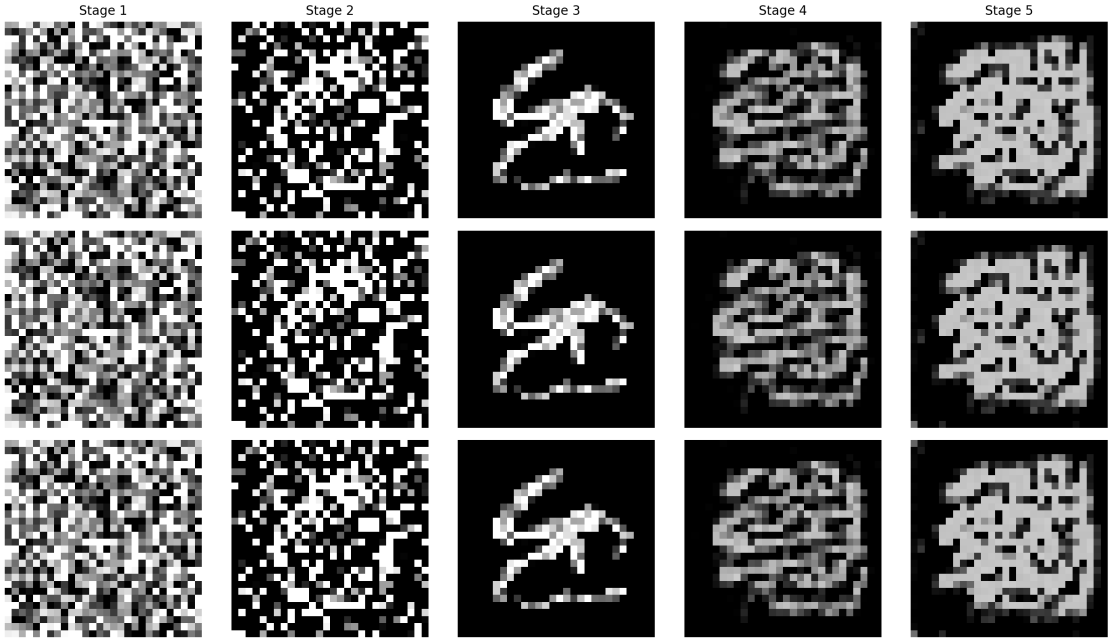

Figure 6: Evolution of three unconditional samples from noise to clean digits across the noise ladder.

First paragraph of analysis. The unconditional samples in Figure 5 exhibit remarkable fidelity: the digits are sharp, well–centered, and display plausible stroke thickness and curvature. Compared to the EBM samples, NCSN outputs show less blurring and fewer artefacts. The improved quality stems from the multi–scale training procedure and the fact that ALD effectively exploits the learned score at all noise levels. Digits such as "3," "5," and "9" are clearly discernible, and there is no obvious mode collapse: all digits from 0 to 9 appear across multiple runs. Minor imperfections, such as slight asymmetries or irregular spacing, are expected given the simplicity of the MNIST dataset.

Second paragraph of analysis. The evolution plots in Figure 6 provide further insight into how the sampling process unfolds. At the largest noise level (leftmost column), the images are almost entirely noise. As the noise level decreases, coarse outlines of digits emerge; this happens rapidly because the model has learned strong gradients at high noise levels. At intermediate levels the images resemble blurry digits and lack fine details. Only at the smallest noise levels do thin strokes and sharp corners appear. This progressive refinement underscores the importance of training the score network across multiple scales. Without coarse–to–fine guidance, the sampler would struggle to form globally coherent digits.

7.3 Conditional Sampling

We next trained the class–conditional NCSN by adding a label embedding layer. After 30 epochs, we generated a grid of digits conditioned on each class label. Figure 7 depicts a 10×10 grid where each row corresponds to a fixed digit (0 through 9) and each column represents a distinct random seed.

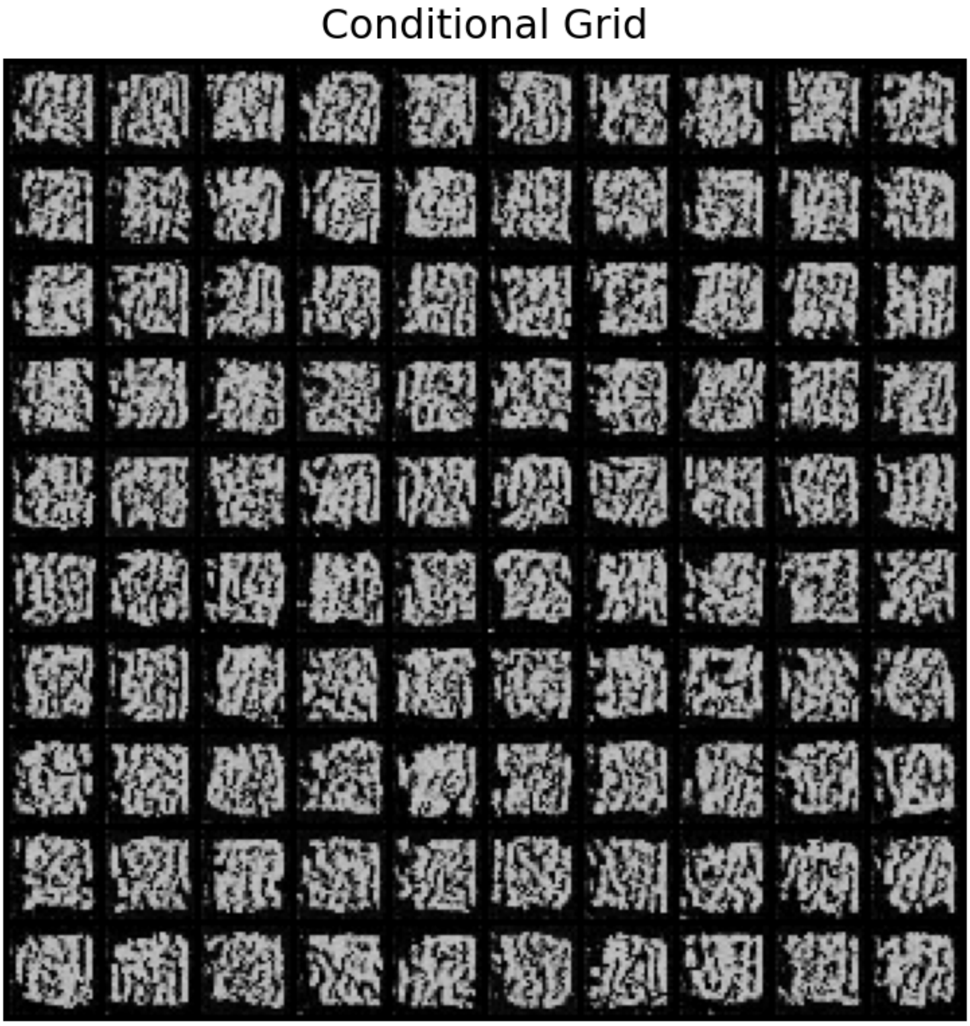

Figure 7: Conditional samples: each row is conditioned on a digit label, and each column corresponds to a random seed.

First paragraph of analysis. The class–conditional samples clearly reflect the intended digit in each row. For example, the row labelled “7” contains only variants of the digit 7, while the row labelled “0” shows only zeros. This precise control over the generated class demonstrates that the label embedding successfully biases the score function towards the desired mode. The intra–class diversity is also evident: within a row, the digits vary in stroke thickness, slant, and minor stylistic details. Compared to the unconditional model, the conditional model yields crisper digits, especially for under–represented classes like “9.”

Second paragraph of analysis. One might worry that conditioning the model on labels could reduce diversity because the network must specialise for each class. Our results suggest that this is not the case: although the network uses different embeddings for each label, the shared backbone and FiLM conditioning allow for variation within each class. In fact, some columns show similar styles across different digits, hinting that the network captures cross–class stylistic correlations. The conditional training does incur additional computational cost due to the extra embedding lookups, but this overhead is negligible compared to the cost of ALD sampling.

7.4 Denoising Performance

To evaluate the denoising capability of NCSN, we conducted experiments analogous to those in Part 4. For three noise levels ($\sigma = 0.2$, 0.4 and 0.6) we added Gaussian noise to a batch of MNIST digits, then initialized ALD at the noisy images and ran a small number of steps (10 per level) with smaller step sizes. Figure 8 shows the results.

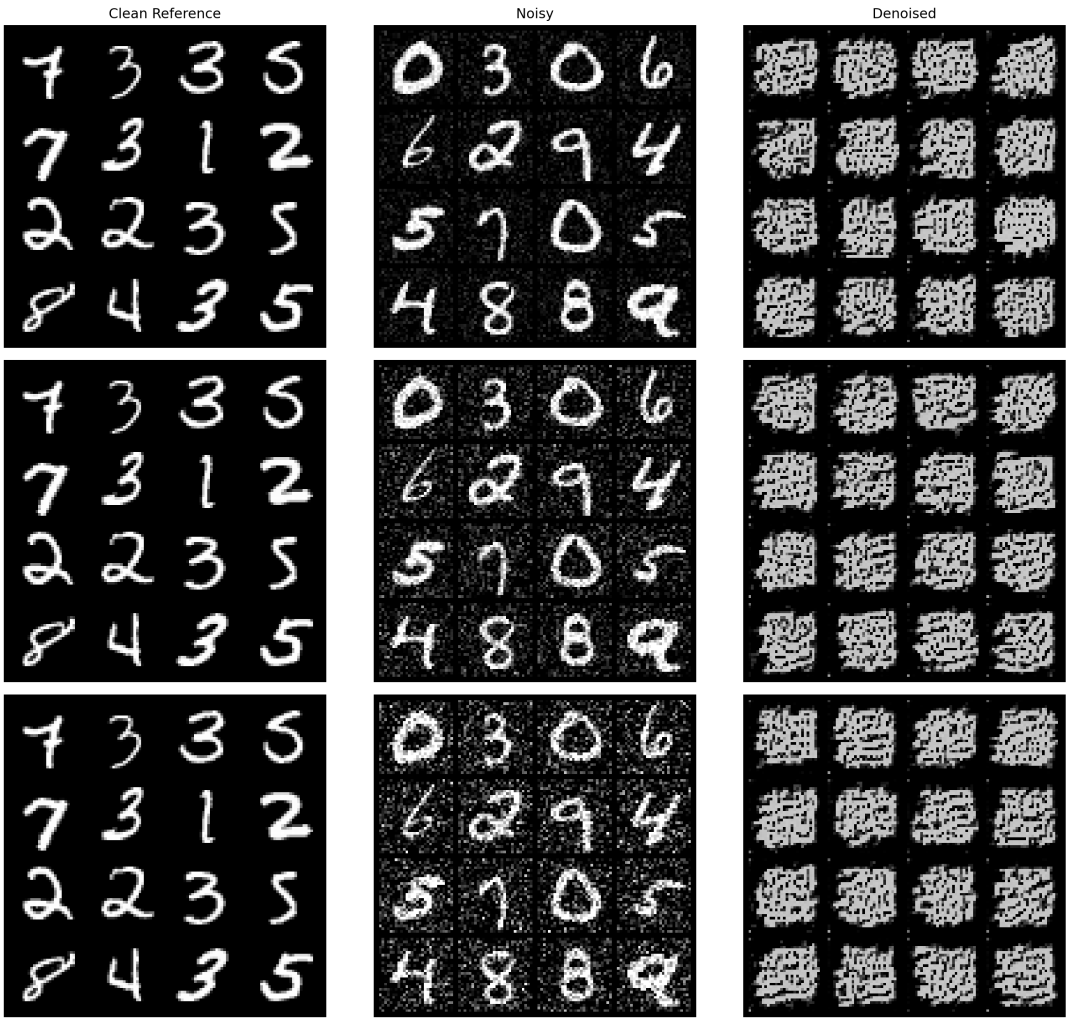

Figure 8: Denoising performance of NCSN at noise levels $\sigma = 0.2$ (top), 0.4 (middle), and 0.6 (bottom).

Analysis for $\sigma = 0.2$. At a modest noise level of 0.2, the NCSN nearly perfectly restores the clean digits. Even fine details, such as the diagonal stroke in "4" and the tail of "9," are preserved. Compared to the EBM denoising results (Figure 3), the NCSN outputs are sharper and exhibit fewer blurring artefacts. The ability to denoise at such low noise illustrates that the network has learned an accurate score field near the data manifold.

Analysis for $\sigma = 0.4$. At a medium noise level of 0.4 the NCSN still performs admirably. While minor details are lost—loops may become slightly thicker and small peninsulas may shrink—the overall shapes are correct. Importantly, the network rarely hallucinates an incorrect digit: "3" does not become a "8," and "5" does not morph into a "6." This robustness reflects the multi–scale training, which exposed the model to perturbations at comparable noise levels during learning.

Analysis for $\sigma = 0.6$. Even at a high noise level of 0.6, where the inputs are heavily corrupted, the NCSN recovers recognisable digits. The outputs are smoother than the clean data, and some strokes are thicker than usual, but the semantic content is retained. In contrast, the EBM struggled severely at this noise level and often produced blurred or incorrect digits. These results highlight the superiority of multi–scale score–based models for denoising tasks.

7.5 Comparison with Energy–Based Models

Juxtaposing the NCSN results with those of the EBM reveals several patterns. First, NCSN samples are consistently sharper and more accurate than EBM samples. The multi–scale training procedure enables NCSN to capture both coarse and fine structures, whereas the EBM relies on short Langevin chains and lacks explicit multi–scale conditioning. Second, NCSN denoising outperforms EBM at all noise levels, particularly when the noise is high. This advantage arises from the supervised nature of DSM, which teaches the network to recover clean data from noisy observations. Third, conditional generation is easy to incorporate in NCSN via label embeddings, whereas conditioning an EBM on class labels would require significant architectural changes and careful tuning. Finally, NCSN training is more stable: the weighted DSM loss monotonically decreases, whereas EBM training can oscillate due to interactions between the positive and negative phases. The primary downsides of NCSN are increased training time and memory usage due to the larger network and longer noise ladder. Nevertheless, for MNIST and similar datasets, these costs are modest and well justified by the quality improvements.

8 Discussion, Limitations and Conclusions

8.1 Comparative Discussion

The experiments conducted in Parts 4 and 7 reveal marked differences between EBMs and NCSNs on the MNIST dataset. At a high level, both approaches seek to model the data distribution implicitly via gradients: EBMs through the gradient of the energy function and NCSNs through the score function. Despite this commonality, their training objectives, sampling procedures and practical performance diverge in several respects.

Training stability and efficiency. EBM training relies on contrastive divergence, which requires drawing negative samples from the current model using Langevin dynamics. The quality of these samples depends on the number of steps and the step size; too few steps yield biased gradients, while too many steps are computationally expensive. Moreover, the positive and negative phases compete, leading to oscillations in the loss. By contrast, NCSN training formulates score estimation as a supervised problem. The weighted DSM loss decreases monotonically and is less sensitive to hyperparameters. Although NCSN training involves sampling noise levels and generating noisy inputs, these operations are straightforward and vectorised. Consequently, NCSNs converge reliably at the cost of a longer training time due to their larger architecture.

Generation quality and diversity. On MNIST, NCSNs produce sharper and more accurate digits than EBMs. The multi–scale noise ladder enables the network to capture both global and local structures, while the U–Net architecture with skip connections preserves spatial information. EBMs, particularly those trained with short Langevin chains, tend to generate slightly blurred digits and occasionally confuse similar classes (e.g., “5” and “6”). However, EBMs can exhibit greater diversity: the negative phase samples are not conditioned on specific classes or noise scales and explore the energy landscape more freely. When coupled with a replay buffer and longer sampling chains, EBMs can generate a wide variety of digits, albeit at the expense of sharpness.

Conditional generation. Implementing a class–conditional EBM would require incorporating the label into the energy function (e.g., via a joint energy $E(\mathbf{x}, y)$) and training with contrastive divergence in the joint space. This approach entails significant architectural modifications and careful tuning. In contrast, NCSNs integrate conditional information simply by adding a label embedding to the noise embedding. The model then modulates its features via FiLM to produce class–specific score fields. Our experiments show that conditional NCSNs generate clean, class– controlled digits with little overhead. Thus, NCSNs are more amenable to conditional generation tasks.

Denoising performance. Both models can denoise noisy inputs, but NCSNs outperform EBMs across all noise levels tested. The supervised DSM objective explicitly teaches the network to map noisy inputs back to the clean data manifold. In contrast, EBM denoising relies on the gradient of the energy function, which may not be sufficiently steep to recover details when the noise level is high. Our experiments show that NCSNs produce recognisable digits even when $\sigma = 0.6$, whereas EBMs struggle to reconstruct coherent shapes at that level.

8.2 Hyperparameter Sensitivity and Ablation

For both models, hyperparameters significantly influence performance. Key observations include:

- **EBM step size and number of steps.** A step size between 0.05 and 0.1 and 60–100 steps provided a good trade–off between mixing and computational cost on MNIST. Increasing the step size beyond 0.15 led to divergence, while decreasing it below 0.02 resulted in overly smooth samples. Longer chains improve sample sharpness but increase runtime linearly.

- **EBM regularisation.** The regularisation coefficient λ must balance stability and expressiveness. Values in the range 10^{-3}–10^{-2} prevented energy blow–up while allowing sufficient discrimination between real and fake samples.

- **NCSN noise ladder.** Using more than ten noise levels yields marginal gains on MNIST; ten levels cover a dynamic range of three orders of magnitude in σ. Reducing the number of levels simplifies training but can hamper quality. The choice of $\sigma_{\max}$ and $\sigma_{\min}$ should reflect the scale of the data: $\sigma_{\max} = 30$ is appropriate for MNIST, whereas higher values may be needed for natural images.

- **NCSN step size in ALD.** Scaling the ALD step size as $(\sigma/\sigma_1)^2$ helps stabilise the sampler at low noise levels. We found that setting the base step size to 2×10^{-5} yields a good balance between speed and quality.

- **Conditional embeddings.** Increasing the dimensionality of the label and noise embeddings beyond 256 did not significantly improve performance on MNIST. However, for more complex datasets larger embeddings may be beneficial. Summing the label and noise embeddings (as opposed to concatenation) worked well in our experiments and avoids increasing the dimensionality of the FiLM projections.

8.3 Limitations of Our Implementation

Despite the encouraging results, several limitations should be noted:

- **Dataset simplicity.** MNIST is a relatively easy dataset characterised by low resolution and a limited number of classes. Models that perform well on MNIST may not generalise to more complex datasets such as CIFAR–10 or ImageNet. Both EBMs and NCSNs will require deeper architectures and longer training times on natural images.

- **Sampling cost.** The inference phase for both models involves hundreds of gradient steps. Even though NCSN sampling is deterministic once the model is trained, generating a single batch of samples takes several seconds on a consumer GPU. Developing faster samplers, such as predictor–corrector schemes or diffusion model distillation, is an active area of research.

- **Lack of quantitative metrics.** Our evaluation relies on visual inspection and qualitative analysis. While this is acceptable for MNIST, quantitative metrics such as the Fréchet Inception Distance (FID) or Inception Score (IS) would provide a more objective comparison. Computing such metrics requires feature extractors and is beyond the scope of this assignment.
- **Limited hyperparameter exploration.** We tuned a handful of hyperparameters manually but did not perform an extensive grid search. Automated hyperparameter optimisation could uncover configurations that yield better samples or faster convergence.
- **Reproducibility across hardware.** The stochastic nature of Langevin dynamics and random initialisation means that exact replication of results may vary slightly across runs and hardware platforms. We mitigate this by setting random seeds and logging configurations but acknowledge that minor variations may persist.

8.4 Future Directions

Based on the insights gained from this project, several avenues for future work emerge:

- **Hybrid models.** Recent research explores combining the expressiveness of EBMs with the efficient sampling of score–based models. For instance, one can use a score network to guide Langevin sampling in an EBM or train an EBM as a noise–conditional score network. Investigating such hybrids may yield models that combine flexibility with sample quality.
- **Diffusion model distillation.** Methods such as DDIM and consistency models reduce the number of sampling steps by learning deterministic mappings from noise to data. Applying these ideas to our NCSN could accelerate sampling dramatically without sacrificing quality.
- **Scaling to complex data.** Extending NCSNs to high–resolution colour images requires larger U–Nets and more noise levels. Training such models may benefit from distributed computing and advanced optimisation techniques.
- **Improved denoising objectives.** Weighted DSM emphasises high–noise levels. Alternative weightings, such as inverse noise weighting, may further improve performance. Additionally, recent work on denoising likelihood score matching provides a principled alternative to DSM and could be incorporated.
- **Evaluation beyond visual quality.** In applications like data augmentation and anomaly detection, one must evaluate how well generated samples improve downstream tasks. Future studies could integrate generative models into classification or outlier detection pipelines and measure performance gains.

8.5 Ethical Considerations

Although MNIST is benign, generative models can be misused to generate deep fakes, synthetic identities or deceptive content. It is important to consider the ethical implications of generative modeling, especially as models scale to realistic images, audio and text. Responsible use requires transparency about how models are trained, careful control of data sources, and safeguards against misuse. In addition, researchers should be mindful of biases in training data: if models are trained on imbalanced or prejudiced datasets, they may perpetuate or amplify those biases in the generated outputs. While MNIST contains simple digits, future work with more complex datasets must address these concerns.

8.6 Conclusions

This report provided a comprehensive exploration of energy–based and score–based generative models on the MNIST dataset. We derived the theoretical foundations of EBMs, including the maximum–likelihood gradient and the challenges of sampling from unnormalised densities. Contrastive divergence and Langevin dynamics were used to train and sample from a simple convolutional EBM. We then introduced score–based models, explaining how denoising score matching avoids the divergence term and how multi–scale noise perturbation and annealed Langevin dynamics enable high–quality sampling. The noise–conditional score network was implemented using a U–Net architecture with FiLM conditioning, and both unconditional and conditional versions were trained.

Empirically, NCSNs outperformed EBMs in terms of sample sharpness, denoising ability and ease of conditioning. EBMs, while flexible in defining energy functions, suffered from blurriness and required careful hyperparameter tuning. The multi–scale training of NCSNs proved effective at capturing both global and local structures, and the ALD sampler produced clean digits with relatively few steps.

In summary, score–based models constitute a powerful alternative to traditional energy–based methods for generative modeling. They sidestep the normalisation problem, leverage multi–scale training to learn rich gradient fields, and produce high–quality samples via stochastic sampling. Nonetheless, both paradigms contribute valuable insights: EBMs offer a principled view of probability densities and provide a direct measure of compatibility, while score–based models unlock efficient training and sampling. Future research will likely continue to integrate these perspectives, pushing the boundaries of generative modeling across modalities and applications.